

Optimizing Qwen2.5-Coder Decode on Tenstorrent Blackhole

Two Wins, One Dead End, and What We Learned

Rassul Magauin Assylzhan Khamiyev
Mohammed Rashed Ali Yahmoor Alshehhi
Sultan Akimaliyev Hamad Khalifa Alyahyaee

April 2026

Mohamed bin Zayed University of Artificial Intelligence
AI Systems — Prof. Abdulrahman Mahmoud

Where we started

Direction B — optimization. Take Qwen2.5-Coder on Tenstorrent Blackhole and make it faster.

- Base: `tt_transformers` already supports the Qwen2.5 family.
- Hardware: 4× Blackhole p150 in tensor-parallel mesh.
- Method: profile, rank, fix, re-profile. Repeat three times.

Spoiler

We tried three optimizations. Two worked. One made the model *slower*. The failure is the most interesting result.

The Machine

Blackhole p150 in one slide

- 130 Tensix cores in an 8×10 compute grid
- Each core: FPU (matrix), SFPU (vector), 5 RISC-V CPUs, 1.5 MB L1 SRAM
- 8 DRAM banks, 2D NoC
- 3 math-fidelity modes:
 - LoFi — 4 TFLOPS, drops 4 mantissa bits
 - HiFi2 — 2 TFLOPS, lossless on BFP8
 - HiFi4 — 1 TFLOPS, full BF16

tt-metal programming model

Each op = a program:

- reader kernel pulls tiles
- compute kernel runs FPU/SFPU
- writer kernel pushes tiles

Circular buffers in L1 stitch them together.

Fusion happens at the *pipeline* level, not “one big kernel”.

The workload

Qwen2.5-Coder-7B, architecturally identical to Qwen2.5-7B-Instruct:

- 28 decoder layers, hidden 3584, MLP intermediate 18944
- 28 query heads, 4 KV heads (GQA), $\text{rope_theta} = 10^6$
- SwiGLU MLP: $\text{down}(\text{silu}(\text{gate}(x)) * \text{up}(x))$

Config we profile:

batch 1, 1024-context, 10 decoder layers, paged attention, 2 generated tokens. Tracy profiler with `-a device_kernel_duration`.

Why 10 layers

Compile time at 28 layers ~ 6 min per run. At 10 layers we get comparable op-level breakdowns in under 2 minutes and can iterate.

Baseline: Where Does the Time Go?

Baseline profile, stock recipe

Operator / group	Time (ms)	Calls	%
TopK (sampling, 1-core)	21.82	4	52.2
Matmul (all linears)	15.46	180	37.0
ReduceScatter + AllGather (CCL)	2.01	123	4.8
BinaryNg (eltwise)	0.69	124	1.7
LayerNorm / RMSNorm	0.37	63	0.9
SDPA decode + RoPE + reshapes	0.16	30	0.4
Sampling	0.11	4	0.3
Other	1.15	-	2.7
Total	41.77	928	100.0

Qwen2.5-Coder-7B decode, 4-way Blackhole TP, device 0, two trace-replay sessions totalled.

Two things jump out

TopK at 52%

Only 4 calls (one per generated token).
5.45 ms per call.

The fused SDPA kernel on 30 calls is **under half a millisecond** total.

A single TopK call takes $11\times$ longer than *all* attention kernels combined.

Plan

Fix TopK first (it is the biggest number). Then tackle the MLP. Leave attention alone — the fused kernels are already doing their job.

Matmul at 37%

180 calls. MLP block (FF1+FF2+FF3) is the largest slice.

Attention (QKV + output proj): ~ 1.3 ms.

MLP (gate + up + down): ~ 3.8 ms.

The TopK smell test

```
Op: TopKDeviceOperation
Input shape: [1, 1, 32, 38016]
k: 32
CORE COUNT: 1 <-- one Tensix core out of 130
DEVICE KERNEL DURATION [ns]: 5,455,012
```

Input width 38016 = Qwen vocab 152,064 / 4 devices. Not a power of two. And reduction/topk/ (under ttnn/cpp/ttnn/operations/) has a multi-core bitonic sort implementation.

Hypothesis: pad the per-device vocab up to $65,536 = 2^{16}$. Kernel flips to the multi-core path. Done.

One-line change in model_config.py. What could go wrong?

Attempt 1: Pad the Vocab (It Didn't Work)

What we changed

In `model_config.py`, replace

```
self.padded_vocab_size = 128 * 1024 if self.is_galaxy else None
```

with a rule that rounds *per-device* vocab up to the next power of two. For Qwen2.5-7B on 4 devices: 38016 $\rightarrow$ 65536.

The LM head gets a wider output. TopK gets more input, but on a presumed-parallel kernel. Compile, re-profile, celebrate.

What actually happened

	Baseline	Padded vocab
TopK total time (ms)	21.82	40.16
TopK time per call (μ s)	5,455	10,039
TopK CORE COUNT	1	1
Matmul total (ms) (LM head n grew)	15.46	11.84
<i>Total device (ms)</i>	<i>41.77</i>	<i>56.82</i>

TopK got 84% slower. End-to-end 36% slower.

Why it failed

The kernel **never switched paths**. CORE COUNT stayed at 1.

We just fed the same single-core kernel $1.72\times$ more data.

Lesson

On `tt-metal`, kernel dispatch is chosen by the op's *device operation* class based on the full tensor spec — **not by input shape alone**.

“Round up to a friendlier shape and hope for a faster code path” is a GPU reflex. It does not transfer.

Reverted (commit `f6de95bb02`). Cleared the stale `output_1m_head_*` tensor cache and re-profiled to confirm.

What a real TopK fix would take

- Parallelize `reduction::topk`'s single-core path, or
- Fuse TopK into `SamplingDeviceOperation`.

Both are C++ kernel work. Not a Python-level configuration tweak. We filed it as future work and moved on.

Side observation

The stock Qwen2.5-7B recipe ships with TopK at 45–52% of per-token device time. **Every user of this recipe** is paying that tax, not just us.

Attempt 2: MLP HiFi4 $\rightarrow$ HiFi2

Why HiFi2 is free here

- HiFi4 = 1 TFLOPS on BF16, retains full mantissa
- HiFi2 = 2 TFLOPS, drops extra mantissa bits *during the dot product*
- When both operands are BFP8, HiFi2 drops nothing that was there

Stock performance_decoder_config.json for Qwen2.5-7B:

```
"LI_FF1_FF3": "HIFI4",  
"LI_FF2": "HIFI4",
```

→ change both to "HIFI2" in all 28 layers. 56 field edits, otherwise a no-op.

Result

	Baseline	+HiFi2	Δ
Matmul total (ms)	15.46	10.45	-32.4%
TopK (ms)	21.82	21.82	0%
CCL (ms)	2.01	2.01	0%
BinaryNg (ms)	0.69	0.69	0%
<i>Total device (ms)</i>	<i>41.77</i>	<i>36.75</i>	<i>-12.0%</i>

- Biggest single win in the project: **32% off matmul time**.
- Accuracy: ci-token-matching passes. 2-layer smoke test decodes coherently.

Attempt 3: Fuse Gate + Up Projections

The idea

SwiGLU: $\text{silu}(\text{gate_proj}(x)) * \text{up_proj}(x)$

Two matmuls share the same input x :

- gate_proj : $[3584] \rightarrow [5120]$ per device
- up_proj : $[3584] \rightarrow [5120]$ per device

Concatenate the weights: single matmul $[3584] \rightarrow [10240]$, then slice.

tt-metal-specific reason: `dram_shard_core_grid_for_k_and_n` picks a different grid for the wider n . We predicted 4 cores $\rightarrow$ 8 cores $\rightarrow \approx 2\times$ matmul speedup.

Implementation, in one place

```
# mlp.py __init__
self.fuse_gate_up = (not args.is_galaxy) and (prefetcher is None)
# build w_gate_up next to w1, w3 (keeps prefill path working)
w_fused = torch.cat([w1_3d, w3_3d], dim=-1)\
    .reshape(args.dim, 2 * args.hidden_dim)

# mlp.py forward()
use_fused = self.fuse_gate_up and mode == Mode.DECODE
if use_fused:
    fused = ttnn.linear(x, self.w_gate_up, ...)
    fused = ttnn.to_memory_config(fused, ttnn.L1_MEMORY_CONFIG)
    w1_out = fused[..., :H] # slice
    w3_out = fused[..., H:] # slice
    # then existing ttnn.mul with activations=[SILU]
```

Guarded on DECODE so prefill, Galaxy, and Llama-prefetcher paths are untouched. Took two passes to get right — prefill binary_ng barfed the first time.

Result (on top of HiFi2)

	HiFi2 only	+Fusion	Δ
Matmul total (ms)	10.45	10.28	-1.6%
calls	180	150	
BinaryNg (ms)	0.69	0.29	-58%
Slice (new) + reshard	0.08	0.24	+0.16
<i>Total device (ms)</i>	<i>36.75</i>	<i>36.38</i>	<i>-1.0%</i>

The matmul gain was smaller than we predicted. The doubled core grid helps, but decode batch-1 matmul is DRAM-bandwidth-bound on the weight read — more cores wait on the same DRAM bus.

The *real* win showed up somewhere we didn't expect: the downstream `binary_ng` eltwise got 58% cheaper because it now consumes L1-interleaved instead of L1-width-sharded inputs.

End-to-End

All three attempts, side by side

Configuration	Total (ms)	Std	vs. baseline
Baseline	41.78	0.008	—
+ HiFi2	36.78	0.009	−12.0%
+ HiFi2 + Fusion	36.38	0.005	−12.9%
TopK padding (failed)	56.80	0.032	+36.0%

- Numbers are mean of 5 repeated runs. Std is in *microseconds*; deltas are in milliseconds. Signal-to-noise ratio 150:1 to 3,000:1 — these are not measurement noise.
- Per-iter decode: 10.4 ms → 9.1 ms. Throughput at 10 layers: $\sim 96 \rightarrow \sim 110$ tok/s/user.
- ci-token-matching passes. No generation-quality regression.

Does the speedup hold at other context lengths?

	seq=512	seq=1024	seq=2048
Baseline	41.78	41.77	41.78
+ HiFi2	36.76 (−12.00%)	36.77 (−11.98%)	36.78 (−11.96%)
+ HiFi2 + Fusion	36.40 (−12.88%)	36.39 (−12.88%)	36.41 (−12.85%)

- Speedup is **flat across input shapes** (variation < 0.05pp).
- Absolute baseline is also flat — at decode batch 1 the cost is dominated by DRAM weight reads and the 1-core TopK; neither scales with cached-context length.
- SDPA does grow with seq_len, but starts at 0.4% of total, so the growth disappears under the microsecond noise floor.

Lessons

What we'd tell someone starting out

1. **Profile before optimizing.** Our biggest “fix” turned out to be against a one-core sampling op nobody was looking at. Without Tracy we'd have spent the semester tuning attention.
2. **On `tt-metal`, shape is not dispatch.** Padding inputs to look friendlier is a CUDA reflex. Here the *device operation* class chooses the kernel, and shape is only one input to that decision.
3. **Expected wins are often replaced by unexpected ones.** Fusion's matmul payoff was small. The payoff that actually mattered was a cheaper eltwise on interleaved operands. Re-profile after every change.
4. **Negative results are worth writing up.** Our TopK padding attempt is something somebody else was eventually going to try. Now they don't have to.

- **Parallelize TopK.** Largest single number in the profile. C++ kernel work.
- **DRAM weight prefetcher for Qwen.** Exists for Llama-3.1-8B. Would overlap weight fetch with compute for the MLP matmuls and help the bandwidth-bound fused matmul we built.
- **Async AllGather + matmul.** 4.8% CCL overhead still sits in the critical path. Reference implementation: `experimental/ccl/llama_all_gather_matmul_async/`.

Thank you.

Questions?

Backup: Reproducing a profile

```
cd /home/rassulmagauin/tt-metal
source python_env/bin/activate
export HF_MODEL=/home/rassulmagauin/models/Qwen2.5-7B-Instruct
export TT_METAL_HOME=/home/rassulmagauin/tt-metal
export MESH_DEVICE=P150x4

python3 -m tracy -p -r \
  -o generated/profiler/reports/<name> \
  --check-exit-code -a device_kernel_duration -t 5000 \
  -m 'pytest models/tt_transformers/demo/simple_text_demo.py \
    -k "device-perf and performance" \
    --num_layers 10 --batch_size 1 --max_seq_len 1024 \
    --max_generated_tokens 2 --mode decode --paged_attention 1 -x'
```

Analyze with `tt-perf-report` on the emitted `ops_perf_results_*.csv`. ~90 s wall time per run on a warm cache.

Backup: Where the code lives

Changes are on top of tt-metal commit a4d8480d3e. Three commits, two of them shipped:

- 965e0f4622 — HiFi4 → HiFi2 for MLP FF1/FF2/FF3 in `performance_decoder_config.json` (all 28 layers).
- e05a044c4f — Gate/up projection fusion in `models/tt_transformers/tt/mlp.py`. Gated on decode mode; prefill and Galaxy configs use the original two-linear path.
- f6de95bb02 — Revert of the TopK vocab padding in `model_config.py`. Kept for provenance.

Backup: Compile-cache variance

Back-to-back runs of architecturally identical models showed $\sim 44\%$ variation in matmul mean time per op, traced to compile-cache state.

For this report, all three comparison runs (Tables 2, 3, 4 of the paper) were taken with warm caches inside the same shell session. Deltas are stable under repeated measurement.

Raw CSVs are under
generated/profiler/reports/
qwen25_7b_decode_*/
in the tt-metal working tree.